Self-organization and Spatial Abstractions in Modern Distributed Computing

Franco Zambonelli
franco.zambonelli@unimore.it

Agents and Pervasive Computing Group
Università di Modena e Reggio Emilia
www.agentgroup.unimore.it

Università di Modena e Reggio Emilia

WETICE 2004

Agents and Pervasive Computing Group

Outline

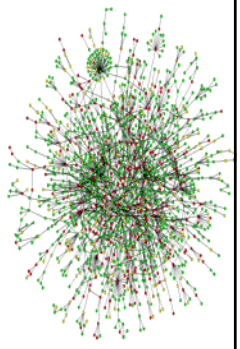
- Key characteristics of Modern Computing Scenarios
  - Large-size, network dynamics, situatedness
- From transparent Distributed Computing to Spatial Computing
  - Towards a paradigm change…
  - From network to space
- Spatial Computing abstractions
  - Shaping space
  - Navigating space
- Spatial computing and self-organization
  - Very good friends…
- Spatial computing researches at the "Agents and Pervasive Computing Group"
  - Computational fields
  - "Spatial" programming models and middleware

Motivations

- Modern distributed computing scenarios are very complex
  - Large size
    - high number of distributed components
  - Network Dynamics
    - decentralized deployment
    - mobile and ephemeral nodes
  - Situatedness
    - activities are related to the environment in which components situates and execute
- Novel approaches to distributed systems development and management are required
  - Decentralized management
  - Self-* (self-organization, self-repair, self-monitoring…)
  - Can be promoted by "Spatial Computing" approaches
- The above considerations apply for a variety of diverse scenarios…
  - Micro scale, medium scale, macro scale

The Macro Scale

- "Macro" in terms of:
  - Geographical distribution
  - High-end computing systems
- Scenarios
  - Routers and IP nodes
  - The Web network
  - P2P Networks & Grid
- Millions of interconnected computer-based
  - or autonomous software components
- Characteristics:
  - Large size: millions of nodes
  - Dynamic Networks: decentralized deployment, ephemeral nodes and services
  - Situatedness: computational resources



The Medium Scale

- "Medium" in terms of
  - Medium-end computers (smart hardware, smart artifacts, PDAs, etc.)
  - Geographical distribution (a town or a building)
- Scenarios
  - Intelligent cities and building
  - Mobile ad-hoc networks
  - Wearable networks
- Key characteristics
  - Large size: hundreds to thousands of components
  - Dynamic networks: mobile nodes, dynamic arrivals of new nodes
  - Situatedness: navigation and actuation in the environment



"INTELLIGENCE CITIES"



A WEARABLE NETWORK

The Micro Scale: Sensor Networks

- "Micro" in terms of
  - Low-end small computer-based devices
  - Local distribution
- Scenarios
  - Pervasive Monitoring
  - Health care
  - Child care
- Key characteristics
  - Large size: hundreds to thousands of components
  - Dynamic networks: random development, short like nodes
  - Situatedness: ***sensing*** the physical world



"SMART DUST" *(from IEEE CiSE, 1997)*



THE BERKELEY MOTES

The Micro Scale: Smart Materials

- From "Micro" to "Nano"
  - Sub-millimeter scale
  - Local distribution
- Scenarios
  - Self-assembly material
  - Digital swarms
  - Artificial Life
- Key characteristics
  - Millions of components
  - Dynamic networks: "spray development", ephemeral and mobile nodes
  - Situatedness: ***acting*** in real world



AN ARTIFICIAL ANT *(from IEEE Computer, 2001)*



T-1000 *(from "Terminator 2)*



SELF-ASSEMBLING BOEING *(Stoy and Nagpal, 2004)*

The KEY Question

- Despite the different scales involved
  - All the above are scenarios of distributed computing
  - All the above shares the same key characteristics

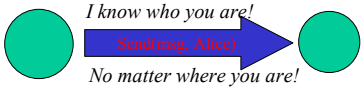
Accordingly:

- A general purpose approach may be very likely possible
  - Same programming paradigm
  - Same set of programming abstractions
  - Same engineering methodologies

What will it be?

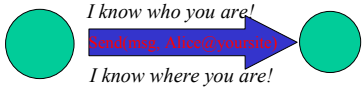
A Historical Perspective (1)

- 80's
  - Transparent Distributed Computing
    - Forget about distribution
    - Components not aware of the distributed nature of the scenario
  - Suitable for
    - Small size systems (transparency hardly scales)
    - Closed systems (mostly static interaction patterns)
    - Not situated applications (no environmental abstractions)



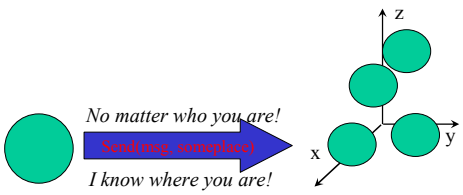
A Historical Perspective (2)

- 90's
  - Network-aware Computing
    - Components aware of the distributed nature of the operational environment
    - E.g., an agent accesses IP_1, IP_2, IP_3….
  - Suitable for
    - Large-size open systems (the costs of distribution can be taken into account)
    - Does not deal with dynamic networks and dynamic operational environment (the network is the environment)
    - Requires explicit exception handling and heavyweight middleware (discovery services)



From Network-aware to Spatial Computing

- From "network" to "space"
  - The operational environment is no longer a network
  - Components perceive their being immersed in some sort of physical or virtual metric space
  - And rely on such a space for all activities…

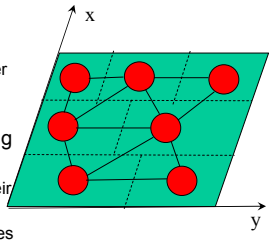


Spatial Computing

- A distributed operational environment is abstracted in terms of an abstract metric space
  - The space has some sort of "structure"
    - An N-dimensional Euclidean space, a torus, a complex manifold,
  - Active entities have a "position" in that space
    - I am at (x_0, y_0, z_0)
- Components' activities related to some sorts of navigation in that space
  - Perceiving the local properties of space
    - I hear data coming from North…
  - Moving themselves (or making data move) in space
    - Let's go to (x_1, y_1, z_1) , send a message to (x_2, y_2, z_2)
  - Changing the properties of space

Example: Structured Overlays

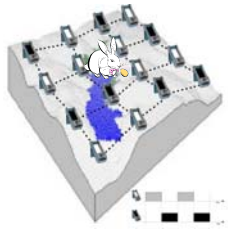
- A search space is abstracted as a Euclidean space
  - e.g., hash mp3 names into coordinates
- Peers occupy a portion of the space
  - And network with each other accordingly to spatial neighborhood relations
- Peers do spatial computing activities:
  - takes care of the data in their own portion of the space
  - Route messages and queries based on geographic routing



Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Example: Sensor Networks

- Deployed in an environment to perform monitoring activity
  - e.g., is there a rabbit in the park? Where is it going?
- Each sensor monitors a specific portion of the space
  - They must know where they are in space
- Coordination between sensors is related to their position in space
  - e.g., if the rabbit is going North: "to all northern sensors: stay tuned!"



Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Advantages of Spatial Computing

- It solves the key problems of modern distributed computing scenarios
  - **Large size**
    - the "size" of the space (and of the network) does not matter
    - only the topology – structure – of the space matter
  - **Network dynamics**:
    - the network structure is masked behind the space
    - the arrival, dismissing, and the identity of network components does not influence application activities
    - Provided that the structure of space and the capability of navigating in the space is preserved
  - **Situatedness**:
    - a spatial computing model is explicitly situated in space
    - whether a virtual computational space or a physical space
- And it is intrinsically "self-organization-oriented"

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Spatial Computing and Self-organization

- What about self-organization?
  - And self-* properties in general?
  - Self-configuring, self-healing, self-organizing, self-managing
- Definition of self-organizing distributed system:
  - A system which is capable of
  - ***autonomously*** (i.e., without human supervision)
  - ***adaptively*** (i.e., without strong a priori assumption about the operational environment) and
  - ***dependably*** (i.e., tolerating dynamics of the operational environments)
  - configuring its computational activities towards the achievement of a specific application goal

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Self-organization for Spatial Computing

- Spatial computing calls for several "self" mechanism…
  - A system must be able to self-organize the space structure
  - As a basic support for spatial application-level activities
  - ("middleware level" self-organization??)
- Components must find their position in space:
  - self-localization
    - Radio triangulation, RF-ID tags, etc.
- A regular metric structure must emerge in space
  - Self-configuration of a structured overlay
    - Recursive self-localization, Self-stabilizing algorithms
- The regular structure must be preserved despite dismission of network nodes
  - Self-healing of the network structure
    - Self-assessment of nodes positions, Dynamic load balancing
- New nodes must somehow get to existence in space
  - Self-advertising & Self-discovery of network partners
    - Broadcast, gossiping, epidemic algorithms

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Spatial Computing for Self-organization

- At the application level, doing distributed computing implies coordinating distributed activities
  - Requires context-awareness: components needs somehow to know "what's around"
- In spatial computing
  - Components perceive local properties of space
    - Pheromone concentration, gravity, slopes, etc.
  - Components "navigate in space"
    - With mechanisms depending on the structure of the space
    - And based on the locally perceived properties
  - The structure of the space and the position of components in it suggest components what to do (context-aware coordination)
  - Activity of components influence the space and it local properties → feedback → self-organization
- In addition…
  - Physical and biological phenomena of self-organization occur in space → are forms of spatial self-organization
  - Thus useful sources of inspirations…

The Spatial Computing Stack

- Towards a reference architecture for spatial computing
- Physical Level
  - Getting to existence in space
  - E.g., Broadcast, RF-ID identification, radar-like localization, etc.
- Structure Level
  - Shaping the virtual space
  - E.g., geographical self-localization, spanning trees, overlay networks, computational fields
- Navigation Level
  - Directing activities in space
  - E.g., Gossiping, Geographical routing, Stigmergy, Directed diffusion, accommodating gravity
- Application Level
  - Exploiting the spatial organization to achieve some high-level application goal in a self-organizing and adaptive way
  - E.g., pattern formation, robot motion, P2P data discovery, etc.

Mechanisms of Spatial Self-Organization

	MICRO SCALE Nano Networks, Sensor Networks, Smart Dust, Computational Self-Assembly, Modular Robots	MEDIUM SCALE Home Networks, MANETs, Pervasive Environments, Mobile Robotics	GLOBAL SCALE Internet, Web, P2P networks, multiagent systems
Application Level	Spatial Queries Spatial Self-Organization and Differentiation of Activities Spatial Displacement Motion Coordination Pattern formation DATA: environmental data	Discovery of Services Spatial Displacement Coordination and Distribution of Task and Activities Motion coordination Pattern formation DATA: local resources and environmental data	P2P Queries as Spatial Queries in the Overlay Motion Coordination on the Overlay Pattern formation (e.g., for network monitoring) DATA: files, services, knowledge
Navigation Level	Flooding Gossiping (random navigation) Geographical Routing (selecting and reaching specific physical coordinates) Directed Diffusion (navigation following sorts of computational fields) Stigmergy (navigation following pheromone gradients)	Computational fields Multi-hop routing based on Spanning Trees Pattern-matching and Localized Tuple-based systems	Flooding Gossiping (random navigation) Metric-based (moving towards specific coordinates in the abstract space) Gossiping (random navigation) Stigmergy (navigation following pheromone gradients distributed in the overlay network)
Structure Level	Self-localization (beacon-based triangulation)	Self-localization (Wi-Fi or RFID triangulation) Definition and Maintainance of a Spanning Tree (as a sort of navigable overaly)	Establishment and Maintainance of an Overlay Network (for P2P systems) Referral Networks and e-Institutions (for multiagent systems)
Physical Level	Radio Broadcast Radar-like localization	Radio Broadcast RF-ID identification	TCP broadcast – IP identification Directed TCP/UDP messages Location-dependent Directory services

Examples

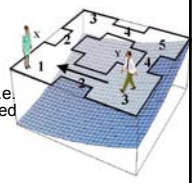
- P2P Overlays
  - **Physical Level**: IP broadcast, or reference server to get initial IP addresses
  - **Structure Level**: ring-shaped spaces (e.g., Pastry) or n-dimensional Euclidean spaces (e.g., CAN)
  - **Navigation Level**: Geographical Routing on the Overlay network, based on DHT (Distributed Hash Table)
  - **Application Level**: Content-based data access and content-based publish-subscribe
- Sensor Networks
  - **Physical Level**: Short-range radio broadcast
  - **Structure Level**: the geographical structure
  - **Navigation Level**: Geographical routing, epidemic algorithms, directed diffusion
  - **Application Level**: spatially coordinating sensing activities

Research Agenda

- Is the proposed "spatial computing stack" meaningful
  - Does the stack abstraction make sense
  - If so, what levels?
- Can a unifying "spatial computing model" be identified
  - Suitable for the micro, medium, and macro scale
  - Useful to model approaches as diverse as ant-based, field-based, structured overlay networks, etc.
  - Simple and general purpose enough?
- How can a such model translate into a set of useful tools?
  - Limited set of usable programming abstractions
  - Middleware infrastructure
- What about non-spatial self-organization?
  - Can it be mapped into some sort of spatial self-organization in any case?
  - Abstract spaces?

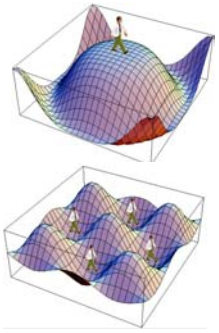
Our Contribution: Field-based Coordination

- Agents' activity and motion driven by local sensing of virtual computational fields spread in space
  - Field actually defines the shape of the space
  - As in gravitational space-time
- Key Concepts
  - **Physical Level**: connecting to a networked environment
  - **Structure Level**: Computational fields (i.e. distributed data structures) are propagated in the environment by agents, and are autonomously updated to reflect environmental dynamics
  - **Navigation Level**: Fields are locally sensed by agents providing spatial information and agents move driven by their slope
  - **Application Level**: Coordinated activities by combining different fields



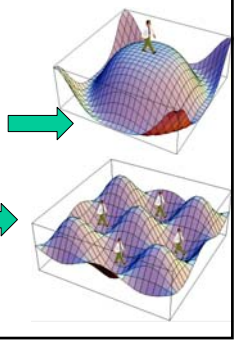
Why Field-based Coordination?

- All the advantages of spatial computing, in addition…
- Fields define a multi-faceted structure of space
  - Components can "sense" different fields in the same space
  - They can locally perceive several interesting properties
  - Expressive context-aware spatial computing
- Components can combine fields and, by navigating the resulting field, can implement a variety of application-specific coordination policies
  - Flexibility
  - General-purpose



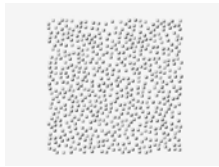
Example: Motion Coordination

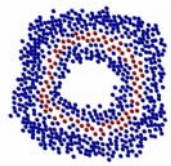
- Can be applied to:
  - Mobile robots, PDA equipped rescue teams, computer-equipped cars…
- Each "component" in an environment generates a "presence" field
  - Updated as the network change and the component moves
- By combining these fields
  - Flocking behavior
  - Coordinated motion



Example: From Motion Coordination to Self-Assembly

- Computational fields can play the role of "morphogen gradients"
  - Diffusion of chemical concentrations along a multi-cellular structure
  - Activation/deactivation of morphogen gradients diffusion based on locally perceived properties
- Can be used to produce a variety of spatial patterns
  - Self-organizing
  - Self-healing





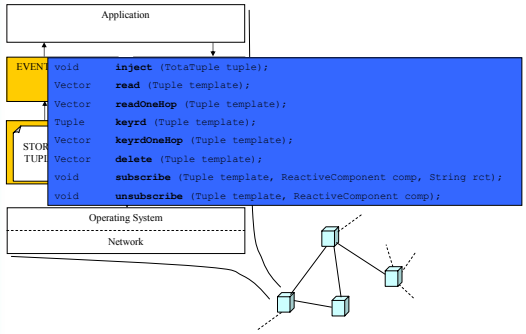
The TOTA Middleware

- *"Tuples On The Air"*
- General purpose (middleware-level) approach to support field-based coordination
  - Generic amorphous network of peer nodes, where each peer runs a local version of the (lightweight) middleware
  - Suitable for both sensor networks, MANETs, P2P networks over fixed network infrastructures,
- Distributed tuples (defining specific types of computational fields) injected and spread in the network:

  T=(C,P)

  Content + ***P***ropagation rule
- Self-Maintenance of tuples' shape to support network dynamics

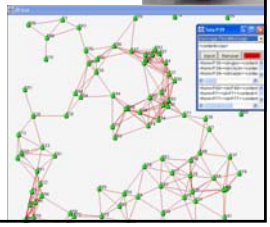
TOTA Middleware Architecture



Implementation

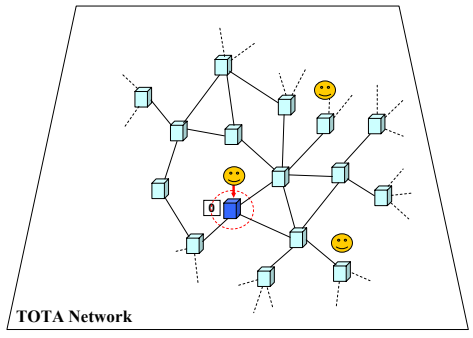
- Implemented in Java (J2ME-CDC) with Compaq Linux IPAQ and 802.11 wireless connection
  - Use broadcast communication
- Implemented a network (MANET) emulator to test in large scale
  - The **same** code installed on simulated nodes, runs also on the IPAQ
- Lego mindstorms + bluetooth
  - In progress





The TOTA Scenario

injection of a tuple



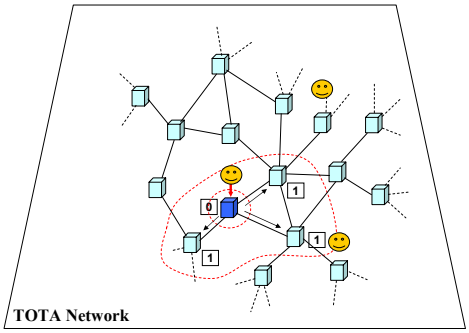
Università di Modena e Reggio Emilia

WETICE 2004

Agents and Pervasive Computing Group

The TOTA Scenario

propagation of a tuple



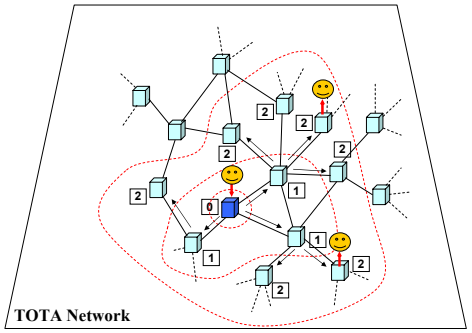
Università di Modena e Reggio Emilia

WETICE 2004

Agents and Pervasive Computing Group

The TOTA Scenario

local sensing of propagated tuple by other agents



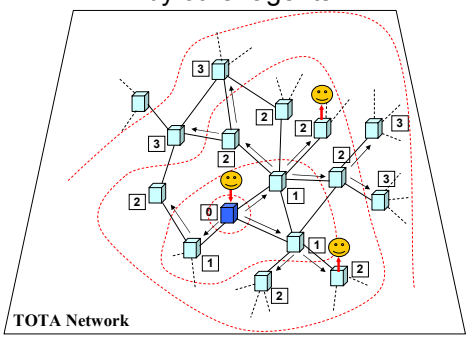
Università di Modena e Reggio Emilia

WETICE 2004

Agents and Pervasive Computing Group

The TOTA Scenario

local sensing of propagated tuple by other agents



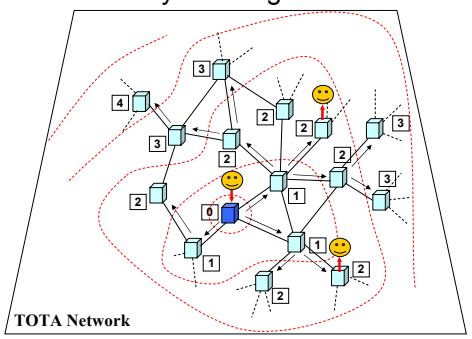
Università di Modena e Reggio Emilia

WETICE 2004

Agents and Pervasive Computing Group

The TOTA Scenario

local sensing of propagated tuple by other agents



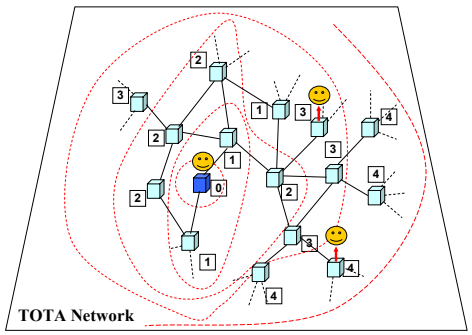
Università di Modena e Reggio Emilia

WETICE 2004

Agents and Pervasive Computing Group

The TOTA Scenario

automatic update of the distributed tuple structure upon topology change



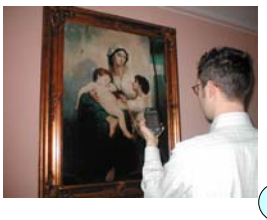
Università di Modena e Reggio Emilia

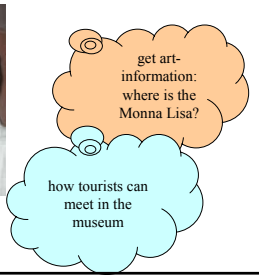
WETICE 2004

Agents and Pervasive Computing Group

Location-dependent Mobile Computing

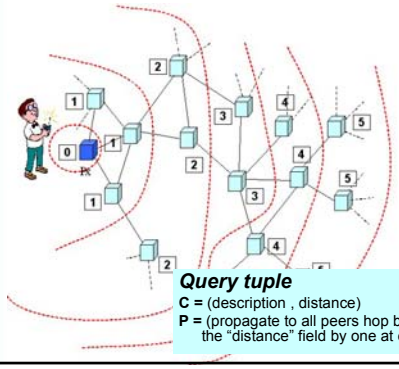
Support a group of tourists visiting a museum





Case Study with TOTA

where is “Monna Lisa”? Propagate a query field



Query tuple

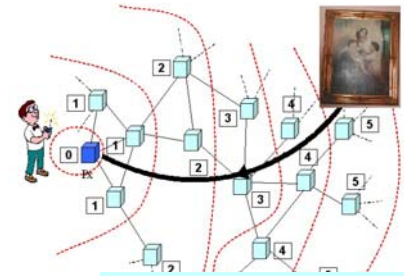
C = (description , distance)

P = (propagate to all peers hop by hop, increasing the “distance” field by one at every hop

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Case Study with TOTA:

Monna Lisa answers following query field downhill



Answer tuple

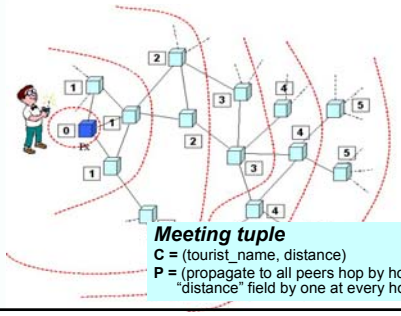
C = (description, location, distance)

P = (propagate following downhill the “distance” of the associated query tuple, incrementing distance value by one at every hop)

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Distributed Motion Coordination

Marco, let’s meet each other! Marco and Franco both propagate a “meeting” field



Meeting tuple

C = (tourist_name, distance)

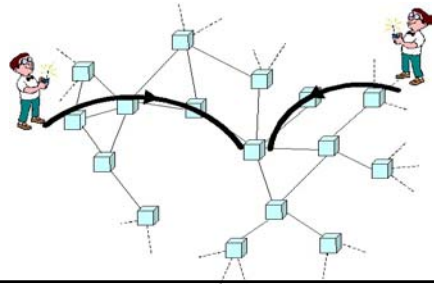
P = (propagate to all peers hop by hop, increasing the “distance” field by one at every hop)

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Case Study with TOTA:

Marco and Franco follow each others’ meeting fields and eventually meet

(important: meeting fields automatic updated as Marco and Franco move…)



Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Generality of Field-based Coordination and of TOTA

- Motion coordination
  - Rescue teams
  - Distributed gaming
  - Swarm Intelligence
- Computational self-assembly
  - Pattern formation in mobile computational particles
  - Modular robots
- Location-aware mobile computing
  - Mobile data & services access
  - Self-localization
- P2P Computing
  - Structured overlay networks

Università di Modena e Reggio Emilia
WETICE 2004
Agents and Pervasive Computing Group

Conclusions

- Spatial computing suitable for modern distributed scenarios
  - Promoting self-organization in large-scale, dynamics, situated networks
- Open issues
  - Look for general purpose models and tools
  - Engineering self-organizing spatial computing applications
- Our Contribution
  - Field-based Coordination
  - TOTA: general-purpose middleware for field-based coordination